Ethereum pipeline. From mnemonic to address

Cryptographic techniques and standards in each step plus an historical overview

Introduction

Ethereum is a decentralized blockchain with smart contract functionality. Ether is the cryptocurrency of the platform. Among cryptocurrencies, Ether is second only to bitcoin in market capitalization.

Authentication in Ethereum is not the one we're used to, username and password: an Ethereum wallet never stores a single password. It starts with a mnemonic, a sequence of 12 or 24 words and, through a chain of transformations, turns it into the addresses used on the blockchain. Each step of this chain relies on a different cryptographic standard, designed at a different time and for a different purpose.

We'll examine that pipeline, from the mnemonic phrase to the checksummed Ethereum address. For every step we'll identify the standard involved (BIP-0039, BIP-0032, BIP-0044, secp256k1, Keccak-256, EIP-55). Then we'll place these technologies in their historical context.

Cryptographic techniques

The pipeline combines six kinds of operations. It is worth defining them precisely before looking at the individual steps.

Key generation: the creation of a new secret from a source of randomness. It is the only step where unpredictability enters the system. In an Ethereum wallet, key generation happens once, when 128 or 256 random bits are drawn from a secure random number generator.

Key derivation: the deterministic computation of one or more keys from existing secret material.

Public-key cryptography: a system based on a pair of mathematically related keys: a private key and a public key. Computing the public key from the private key is easy, while the reverse is computationally infeasible. On elliptic curves, the public key is $Q = kG$, and recovering k from Q is the Elliptic Curve Discrete Logarithm Problem (ECDLP).

Hashing: a cryptographic hash function maps an input of arbitrary length to a fixed-length output. It must be preimage resistant (the input cannot be recovered from the digest), second-preimage resistant and collision resistant, and its output should look random. Keccak-256 is the hash function used to derive Ethereum addresses.

Checksum: redundant information added to data for error detection.

Encoding: a reversible change of representation that involves no secret and offers no security by itself.

The pipeline

Randomness → entropy	Key generation	Secure random number generator
Entropy → mnemonic	Encoding	BIP-39
Mnemonic → 512-bit seed	Key derivation	BIP-39
Seed → master private key	Key derivation	BIP-32
Private key k → public key	Public-key cryptography	secp256k1
Public key → address	Hashing	Keccak-256
Address → checksummed address	Checksum + encoding	EIP-55

BIP-0039

Standard under MIT license, BIP 39's goal is generating the mnemonic and converting it into a binary seed. This seed can be used later to generate deterministic wallets. The idea is to find a way to bring computer generated randomness into a human readable transcription.

The mnemonic must encode entropy in a multiple of 32 bits. The more entropy mnemonic encodes, the more security, but the sentence's length will have an impact.

We define the initial entropy length as ENT and it's 128-256 bits long.

Generation of mnemonic

ENT bits of entropy are generated; the checksum are the first ENT/32 bits of its SHA-256 hash. This checksum is appended to the end of the initial entropy. These concatenated bits are splitted into 11 bits groups each encoding a number from 0 through 2047, this encode is used as an index in the word list. We convert these numbers into words, and the joined words are the mnemonic sentence.

From mnemonic to seed

It's optional to protect the mnemonic with passphrase, if it's not present the empty string "" is used.

To create a binary seed from the mnemonic we use PBKDF2: a key derivation function.

Seed=PBKDF2(PRF, password, salt, c, seedLength)

PRF is the underlying pseudorandom function, in this case HMAC-SHA 512. The master password from which a derived key is generated: our mnemonic. The salt is mnemonic concatenated with passphrase. The number of iteration count is set to 2048 and seedLength to 512 bits.

BIP-0032

From seed to master private key

Assumption: we use as public key cryptography elliptic curve, using the field and curve parameters defined by secp256k1 (see later).

We define a function that derives several child keys from a parent key.

We want to avoid these child keys depending solely on the parent key itself; so we add a chain code to our seed. Chain code is an additional 256 bits of entropy.

From each extended key it's possible to generate up to 2^{31} normal (indices from 0 through 2^{31}-1) and 2^{31} hardened (indices from 2^{31} through 2^{32}-1) child keys.

Child key derivation functions

BIP 32 offers us different ckd functions, we need to generate a private child key (master private key in Ethereum pipeline) from a private parent key (seed).

We take as input parent extended key and an index and the function outputs the child extended key.

These are the steps:

1. We output I, if the child is hardened:
   I=HMAC-SHA512(key= parent chain code, data=0x00 || ser_{256}(parent key) || ser_{32}(i))
   Else
   I= HMAC-SHA512(key= parent chain code, data=ser_p(point(parent key)) || ser_{32}(i))
   With:
   ser_{256} and ser_{32} serialize input as 32 bytes sequence.
   ser_p serializes coordinate pair (x,y) as (0x02 or 0x03)|| ser_{256}(x).
   point returns the coordinate pair resulting from elliptic curve point multiplication of the secp256k1 base point with the integer p.
2. Split I (64 bytes) in I_L and I_R
3. Child key= I_L+parent key (mod n)
4. Child chain code= I_R

BIP-0044

It defines a logical hierarchy for wallets. Here's path level:

m/ purpose' / coin type'/ account' / change / address index

Apostrophe means it's required to use the hardened derivation used in BIP 32.

1. Purpose' : it's set to 44' following the recommendation, since we are using BIP 44 specification
2. Coin type': one master node may be used for unlimited cryptocoins. It's good to create a separate subtree for every of them.
3. Account': this level splits the key space into independent user identities.
4. Change: it's set to 0 if it's an external chain, addresses are visible outside the wallet, 1 otherwise.
5. Index: address numbered from 0 in sequentially increasing manner.

Secp256k1

Cryptographic public key scheme, used to derive public key from master private key

Koblitz curves are a special class of elliptic curves that enable efficient computation. Secp256k1 is $y^2=x^3+7$, the key size is 256 bits. It's designed to work over a Galois finite prime field. It can be used in elliptic curve digital signature algorithms: the private key is 256 bits randomly generated and the public key is sk*G (a predefined point on the curve). Public key is used to generate Ethereum addresses by doing a Keccak-256 hash on the public key and discarding the first 12 bytes. The 20 remaining are the address.

Keccak-256

From public key to address

It's a cryptographic hash function part of the SHA-3 family, it outputs 32 bytes hash.

Comparison between SHA-2/SHA512 and SHA-3/Keccak-256

SHA-2 and SHA-3 are both families of cryptographic hash functions standardised by NIST, but they are built in completely different ways.

SHA-2, which includes SHA-256 and SHA-512, uses the Merkle–Damgård construction: the message is split into blocks that are processed one after the other by a compression function.

SHA-3 is based on Keccak, the winner of a public NIST competition launched in 2007 after attacks on SHA-1. Keccak uses the sponge construction: the input is "absorbed" into a large internal state and the output is then "squeezed" out of it. SHA-3 was not created because SHA-2 was broken, but to provide an alternative with a different internal structure.

EIP-55

Mixed-case checksum address encoding

Implementing a checksum directly onto an Ethereum address is not feasible due to its cryptographic nature, they don't have a common format for an embedded check digit; in contrast the IBAN format incorporates a check digit for error detection. EIP-55 brings a client side checksum without compromising the cryptographic integrity of Ethereum.

Addresses are encoded in hexadecimal, which is case insensitive, EIP-55 is based onto this property.

Here's the rule:

Take the address in lowercase without the initial "0x", hash it using Keccak-256. Walk through the 40 characters, if the corresponding hash character's value is equal or bigger than 8 the character in the address is written in uppercase.

If a user mistypes one character of a checksummed address, the case pattern no longer matches the hash and validation fails.

Statistical properties

Entropy

The whole security of the wallet depends on the initial entropy being uniform and unpredictable. A 12-word phrase encodes 128 bits of entropy plus 4 checksum bits: there are $2048^{12} = 2^{132}$ possible word sequences, but only one in 16 is valid, which leaves 2^{128} valid mnemonics.

BIP-39 checksum

Because the checksum has 4 bits for 12 words, a random sequence of words is valid with probability 1/16, and a random substitution of one word is detected with probability 15/16. With 24 words the checksum has 8 bits, and the probabilities become 1/256 and 255/256.

PBKDF2 and HMAC-SHA512

These key-derivation functions are built from SHA-512 and are expected to behave like random functions.

secp256k1

Scalar multiplication behaves very differently from a hash function. Its outputs look with no visible correlation with k. This structure is essential for digital signatures, but it means that a public key is not a random-looking.

Keccak-256

Hashing the public key removes that structure. The resulting address bits are expected to be uniform and independent. Truncating the output to 160 bits keeps preimage resistance at about 2^{160} operations, while the birthday bound for finding any two colliding addresses is about 2^{80}.

EIP-55

Each letter of the address is capitalised when the corresponding digit of an independent Keccak-256 hash is 8 or higher, so each letter is uppercase with probability 1/2. Since a hexadecimal digit is a letter with probability 6/16, an address contains on average 15 letters, and therefore about 15 check bits. According to EIP-55, a randomly mistyped address passes the check with a probability of about 0.0247%. Addresses written entirely in lowercase or uppercase carry no checksum and cannot be verified.

Historical overview

The Ethereum pipeline combines tools created in over 40 years, by different entities and for different purposes.

Timeline

RSA	1977
SHA-2	2001
BIP-0032	2012
BIP-0039	2013
BIP-0044	2014
SHA-3	2015
EIP-55	2016

The mathematical foundations come first. Public-key cryptography was introduced by Diffie and Hellman in 1976 and made practical by RSA in 1977. Elliptic-curve cryptography, proposed independently by Koblitz and Miller in 1985, offered the same security with much shorter keys.

The next protocols became international standards, not merely designed for cryptocurrencies. SHA-2 was published in 2001, HMAC and PBKDF2 were standardised by the IETF, and the secp256k1 curve appeared in the SEC 2 document in 2000.

Bitcoin, launched in 2009, changed the stakes: losing a private key now meant losing funds, with no authority able to recover them. The Bitcoin community responded with a series of improvement proposals for key management: BIP-32 in 2012, BIP-39 in 2013 and BIP-44 in 2014.

Ethereum, launched in 2015, inherited this whole layer from Bitcoin. It added only: Keccak-256 for deriving addresses and, in 2016, EIP-55 for checksummed addresses. Its choice of Keccak-256 instead of SHA3-256 is a consequence of timing: Ethereum was built while SHA-3 was still being reviewed.

Conclusion

Comparing the steps shows why the pipeline needs different techniques. Hashing and key derivation provide outputs that are statistically indistinguishable from random. Elliptic-curve multiplication provides public key digital signatures, at the cost of algebraic structure. Checksums do not aim at randomness at all, only at detecting human errors with high probability. All of them, finally, depend on the quality of the initial entropy.